

Software Design Document (SDP)

Version 1 Offline Dog Detection, First-N Selection, Persistent Tracking, and Video Labelling Application

Document Status: Software Design Specification

Version: 1.0

Application Type: Local Python Desktop Computer-Vision Application

Primary Function: Process prerecorded video, detect dogs, select the first N distinct dogs, persistently track those dogs, and produce a labelled video for playback and explicit saving.

1. Purpose

This Software Design Document defines the implementation-level design for Version 1 of the offline dog-detection and persistent-tracking application described in the System Architecture Document.

The SDP translates the architectural components and boundaries into concrete Python modules, classes, interfaces, data structures, processing algorithms, state machines, error-handling mechanisms, and testing structures.

The implementation shall remain intentionally small and modular. V1 shall not introduce databases, networking, cloud services, distributed processing, message brokers, or unnecessary framework abstractions.

The central implementation concern is maintaining the distinction between:

```
Detection
  -> Tracker observation
  -> Application target
```

A YOLO detection is not a persistent identity. A tracker ID is also not a permanent application identity. The application owns the final target identity.

2. Design Goals

The implementation shall:

1. Read prerecorded local video files sequentially.
2. Validate input before processing.
3. Run a locally provisioned YOLO model.
4. Keep only dog detections.
5. Confirm candidate dogs before assigning target slots.

6. Select at most MAX_TARGETS distinct dogs.
 7. Assign application-level target IDs.
 8. Track selected dogs across frames.
 9. Handle temporary disappearance.
 10. Avoid replacing selected dogs with later dogs.
 11. Produce annotated frames.
 12. Write frames to temporary video storage.
 13. Make the completed temporary video available for playback.
 14. Save the result permanently only after explicit user action.
 15. Keep the GUI responsive during processing.
 16. Provide bounded error handling.
 17. Provide useful performance metrics.
 18. Permit automated testing through mocked dependencies.
 19. Keep Ultralytics-specific APIs isolated from the rest of the application.
-

3. Implementation Technology

The V1 implementation shall use:

Python
OpenCV
NumPy
Ultralytics
PySide6
psutil
PyTorch

PyTorch is an implementation dependency of the selected YOLO runtime rather than an application-level architectural dependency.

The project shall pin tested dependency versions in the deployment/dependency configuration.

The exact YOLO model file shall be stored locally and shall not be downloaded during normal runtime.

4. Project Structure

The recommended implementation structure is:

```
project/  
├── src/
```

```
├── main.py
├── application.py
├── config.py
├── gui/
│   ├── __init__.py
│   ├── main_window.py
│   ├── playback.py
│   └── state.py
├── video/
│   ├── __init__.py
│   ├── reader.py
│   ├── writer.py
│   └── metadata.py
├── detection/
│   ├── __init__.py
│   ├── detector.py
│   ├── ultralytics_detector.py
│   └── models.py
├── tracking/
│   ├── __init__.py
│   ├── tracker.py
│   ├── ultralytics_tracker.py
│   └── target_manager.py
├── selection/
│   ├── __init__.py
│   └── target_selector.py
├── rendering/
│   ├── __init__.py
│   └── renderer.py
├── processing/
│   ├── __init__.py
│   └── pipeline.py
├── output/
│   ├── __init__.py
│   ├── temporary_output.py
│   └── save_manager.py
├── monitoring/
│   ├── __init__.py
│   └── performance.py
├── logging/
│   ├── __init__.py
│   └── logger.py
├── test/
│   ├── unit/
│   ├── integration/
│   ├── gui/
│   ├── performance/
│   └── e2e/
├── models/
│   └── model.pt
└── test_data/
```

```
├── ...
├── requirements.txt
├── README.md
└── .gitignore
```

The directory structure is an organizational boundary, not a requirement that every module contain a large class.

5. Core Data Models

Application-specific data models shall be defined independently of Ultralytics.

5.1 Detection

src/detection/models.py

Conceptual implementation:

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class Detection:
    class_id: int
    class_name: str
    confidence: float
    x1: int
    y1: int
    x2: int
    y2: int
```

Coordinates shall use:

```
x1, y1 = top-left
x2, y2 = bottom-right
```

and shall be relative to the current frame.

Confidence shall be normalized to the range:

```
0.0 <= confidence <= 1.0
```

Invalid values shall be rejected at the detector boundary.

5.2 TrackObservation

```
@dataclass(frozen=True)
class TrackObservation:
    tracker_id: int
    detection: Detection
    association_confidence: float | None
    state: str
```

confidence in Detection represents detector confidence.

association_confidence represents confidence in temporal association.

These values must never be treated as interchangeable.

5.3 TargetObject

src/tracking/target_manager.py

Conceptually:

```
@dataclass
class TargetObject:
    target_id: int
    class_name: str
    tracker_id: int | None
    latest_detection: Detection | None
    last_bbox: tuple[int, int, int, int] | None
    last_seen_frame: int
    lifecycle_state: str
    visibility_state: str
    association_confidence: float | None
```

Possible lifecycle states:

```
CANDIDATE
ACTIVE
TEMPORARILY_LOST
REACQUIRED
PERMANENTLY_LOST
```

Possible visibility states:

```
VISIBLE
NOT_VISIBLE
```

The implementation may use Python enums rather than strings.

5.4 VideoMetadata

src/video/metadata.py

```
@dataclass(frozen=True)
class VideoMetadata:
    width: int
    height: int
    source_fps: float | None
    frame_count: int | None
    duration: float | None
    frame_count_known: bool
    fps_reliable: bool
```

The implementation shall not assume that OpenCV metadata is reliable.

6. Configuration Design

`src/config.py` shall contain operational configuration.

Example:

```
TARGET_CLASS = "dog"
MAX_TARGETS = 3

DETECTION_CONFIDENCE = 0.40
TARGET_CONFIRMATION_FRAMES = 3

MODEL_PATH = "models/model.pt"
DEVICE = "auto"

TRACKER_TYPE = "botsort"
REID_ENABLED = False

MAX_CONSECUTIVE_READ_FAILURES = 10
MAX_CONSECUTIVE_INFERENCE_FAILURES = 3

PERFORMANCE_LOG_INTERVAL = 5.0

SUPPORTED_EXTENSIONS = (
    ".mp4",
    ".avi",
    ".mov",
    ".mkv",
)

OUTPUT_EXTENSION = ".mp4"

LOG_LEVEL = "INFO"

WINDOW_TITLE = "Offline Dog Detection"
```

The actual values shall be tuned through testing.

Configuration shall not be scattered throughout the implementation.

For example, the processing pipeline should not contain:

```
if confidence < 0.40:
```

It should reference:

```
config.DETECTION_CONFIDENCE
```

7. Application Entry Point

`src/main.py` shall have minimal responsibilities.

Conceptually:

```
def main() -> int:
    initialize_logging()

    application = Application()
```

```

try:
    return application.run()
except Exception:
    logger.exception("Unhandled application exception")
    return 1

```

`main.py` shall not contain the frame-processing algorithm.

The application controller shall own lifecycle coordination.

8. Application Controller

`src/application.py`

The Application class shall coordinate:

```

GUI
Video Reader
Detector
Tracker
Target Selector
Target Manager
Renderer
Writer
Performance Monitor
Save Manager

```

Conceptual interface:

```

class Application:
    def run(self) -> int:
        ...

    def start_processing(self, input_path: str) -> None:
        ...

    def cancel_processing(self) -> None:
        ...

    def shutdown(self) -> None:
        ...

```

The controller shall create dependencies and inject them into the processing pipeline.

It should not implement detection or tracking logic itself.

9. GUI Design

`src/gui/main_window.py`

The main window shall contain, at minimum:

```

+-----+
| Offline Dog Detection |

```

```

| [ Select Video ]
| Selected: example.mp4
|
| [ Process ] [ Cancel ]
|
| Status: Processing...
| Progress: 63%
|
|           Video Playback
|
| [ Play ] [ Pause ] [ Stop ]
|
| [ Save ]
+-----+

```

The GUI shall interact with the controller through high-level commands.

It shall not receive raw YOLO results.

10. GUI State Machine

src/gui/state.py

An enum should represent application UI state:

```

class ApplicationState(Enum):
    IDLE = auto()
    FILE_SELECTED = auto()
    VALIDATING = auto()
    PROCESSING = auto()
    CANCELLING = auto()
    FAILED = auto()
    COMPLETED = auto()
    PLAYBACK = auto()
    SAVING = auto()
    SAVE_FAILED = auto()

```

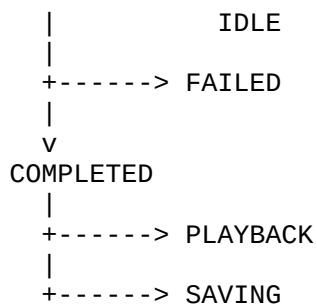
State transitions shall be explicit.

Example:

```

IDLE
|
| file selected
v
FILE_SELECTED
|
| process
v
VALIDATING
|
| valid
v
PROCESSING
|
+-----> CANCELLING
|           |
|           v

```

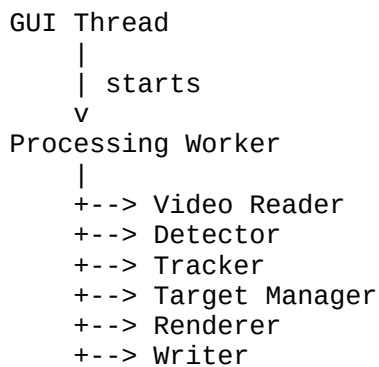


Invalid state transitions should be rejected rather than silently accepted.

11. Background Processing

Video processing must not run directly in the Qt GUI event loop.

The recommended design is:



The worker reports events to the GUI through Qt signals.

Example signals:

```

processing_started
progress_changed
status_changed
target_selected
performance_updated
processing_completed
processing_failed
processing_cancelled

```

The worker shall not directly modify GUI widgets.

12. Video Reader Design

`src/video/reader.py`

Define an application-level reader interface.

Conceptually:

```

class VideoReader(Protocol):
    def open(self, path: Path) -> None:
        ...

    def metadata(self) -> VideoMetadata:
        ...

    def read(self) -> tuple[bool, np.ndarray | None]:
        ...

    def is_open(self) -> bool:
        ...

    def close(self) -> None:
        ...

```

The OpenCV implementation shall wrap:

`cv2.VideoCapture`

rather than exposing `VideoCapture` throughout the application.

13. Input Validation

Validation should occur before the processing worker enters the main frame loop.

Validation sequence:

```

Path supplied
  |
  v
Path exists?
  |
  v
Is file?
  |
  v
Extension supported?
  |
  v
Open VideoCapture
  |
  v
Capture reports open?
  |
  v
Read first frame
  |
  v
Frame valid?
  |
  v
Metadata accepted

```

Validation errors shall be represented using application-specific exceptions or result objects.

For example:

```

class InputValidationError(Exception):

```

pass

The GUI receives a user-readable message while the log contains the technical details.

14. Frame Reading Algorithm

The main loop shall maintain a consecutive read-failure counter.

Conceptually:

```
consecutive_failures = 0

while not cancellation_requested:

    success, frame = reader.read()

    if success and is_valid_frame(frame):
        consecutive_failures = 0
        process_frame(frame)
        continue

    consecutive_failures += 1

    log_read_failure(consecutive_failures)

    if consecutive_failures >= config.MAX_CONSECUTIVE_READ_FAILURES:
        raise VideoReadError(...)
```

The implementation shall distinguish:

EOF

from:

read failure

A clean EOF is normal completion.

A sequence of failed reads before EOF is an error condition.

The exact OpenCV semantics should be tested against representative codecs.

15. Frame Validation

A helper should be implemented in a small module, for example:

```
def is_valid_frame(frame: np.ndarray | None) -> bool:
    if frame is None:
        return False

    if not isinstance(frame, np.ndarray):
        return False

    if frame.size == 0:
        return False
```

```

if frame.ndim not in (2, 3):
    return False

height, width = frame.shape[:2]

return height > 0 and width > 0

```

Additional validation may be added if required by the selected detector.

The validator shall remain independent of YOLO.

16. Detector Interface

src/detection/detector.py

Conceptual protocol:

```

class Detector(Protocol):
    def load(self) -> None:
        ...

    def detect(self, frame: np.ndarray) -> list[Detection]:
        ...

    def close(self) -> None:
        ...

```

The processing pipeline depends on this interface rather than on Ultralytics.

17. Ultralytics Detector

src/detection/ultralytics_detector.py

The implementation shall:

1. Load the configured local model.
2. Configure the requested device.
3. Perform inference.
4. Extract model results.
5. Convert results to **Detection**.
6. Reject malformed detections.
7. Return application-level objects.

Conceptually:

```

ultralytics result
|
v

```

```
Detection Adapter
  |
  v
list[Detection]
```

No Ultralytics result object should escape this module.

18. Dog Filtering

Dog filtering shall occur after conversion to application-level detections.

Conceptually:

```
dog_detections = [
    detection
    for detection in detections
    if detection.class_name == config.TARGET_CLASS
    and detection.confidence >= config.DETECTION_CONFIDENCE
]
```

The target selector should therefore never need to understand the entire COCO class set.

The V1 category is:

dog

19. Detector Error Handling

Detector initialization errors are startup errors.

Examples:

```
model file missing
invalid model
unsupported model
invalid device
runtime initialization failure
```

These should not enter the frame loop.

Per-frame inference errors shall be bounded.

Conceptually:

```
try:
    detections = detector.detect(frame)
    consecutive_inference_failures = 0
except Exception:
    consecutive_inference_failures += 1

    if consecutive_inference_failures >=
config.MAX_CONSECUTIVE_INFERENCE_FAILURES:
    raise InferenceError(...)
```

The exact policy may be stricter if an inference failure compromises tracker state.

20. Tracker Interface

src/tracking/tracker.py

Conceptually:

```
class Tracker(Protocol):
    def initialize(self) -> None:
        ...

    def update(
        self,
        frame: np.ndarray,
        detections: list[Detection],
    ) -> list[TrackObservation]:
        ...

    def reset(self) -> None:
        ...

    def close(self) -> None:
        ...
```

The tracker implementation shall maintain temporal state internally.

21. Ultralytics Tracker Adapter

src/tracking/ultralytics_tracker.py

This module shall isolate:

Ultralytics tracking API
tracker configuration
tracker-specific result extraction
tracker IDs

The rest of the application sees only:

TrackObservation

The adapter may internally use:

BoT-SORT

or:

ByteTrack

based on configuration.

22. Tracker Initialization

Tracker initialization shall occur once per processing session.

The tracker must receive frames sequentially.

It shall not be reinitialized on every frame.

A new processing session shall create or reset the tracker.

This prevents state from one video from contaminating another.

23. Target Selector Design

src/selection/target_selector.py

The selector owns the first-N policy.

Conceptually:

```
class TargetSelector:
    def __init__(
        self,
        max_targets: int,
        confirmation_frames: int,
    ):
        ...
```

Responsibilities:

- candidate creation
- candidate confirmation
- candidate ordering
- target-slot consumption
- duplicate prevention

It does not perform temporal tracking itself.

24. Candidate Tracking

A candidate structure may be implemented as:

```
@dataclass
class Candidate:
    tracker_id: int
    first_seen_frame: int
    consecutive_observations: int
    latest_observation: TrackObservation
```

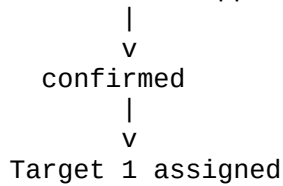
The candidate must be associated with the same tracker identity for the required confirmation period.

Example:

TARGET_CONFIRMATION_FRAMES = 3

Processing:

Frame 10: tracker 5 appears
Frame 11: tracker 5 appears
Frame 12: tracker 5 appears



A one-frame detection does not consume a target slot.

25. Candidate Confirmation Algorithm

Conceptually:

```
def update_candidate(observation, frame_index):
    candidate = candidates.get(observation.tracker_id)

    if candidate is None:
        candidates[observation.tracker_id] = Candidate(
            tracker_id=observation.tracker_id,
            first_seen_frame=frame_index,
            consecutive_observations=1,
            latest_observation=observation,
        )
        return None

    candidate.consecutive_observations += 1
    candidate.latest_observation = observation

    if candidate.consecutive_observations >= confirmation_frames:
        return candidate

    return None
```

The candidate should be removed/reset when the tracker no longer considers it a valid continuous candidate.

26. First-N Assignment Algorithm

The application shall enforce:

```
len(targets) < config.MAX_TARGETS
```

before assigning a new target.

Conceptually:

```
if (
```

```

        confirmed_candidate
        and not target_manager.contains_tracker(
            confirmed_candidate.tracker_id
        )
        and target_manager.target_count < config.MAX_TARGETS
    ):
        target_manager.create_target(...)

```

Once a target is created, that target slot is reserved.

A later dog must never overwrite it.

27. Deterministic Simultaneous Selection

If multiple candidates become confirmed in the same frame, they shall be ordered deterministically.

Recommended ordering:

1. Horizontal bounding-box position
2. Tracker ID

Horizontal center:

```
center_x = (x1 + x2) / 2
```

Then:

```

sorted(
    candidates,
    key=lambda candidate: (
        candidate.latest_observation.detection.x1,
        candidate.tracker_id,
    ),
)

```

This prevents framework iteration order from determining target identity.

28. Target Manager

src/tracking/target_manager.py

The target manager owns application-level target identity.

Conceptual API:

```

class TargetManager:
    def create_target(self, observation) -> TargetObject:
        ...

    def update_target(self, target_id, observation) -> None:
        ...

    def mark_lost(self, target_id) -> None:
        ...

```

```

def attempt_reassociation(self, observation):
    ...

def targets(self) -> list[TargetObject]:
    ...

def target_count(self) -> int:
    ...

def contains_tracker(self, tracker_id: int) -> bool:
    ...

```

Target IDs should be generated by the application:

```

Target 1
Target 2
Target 3

```

rather than copied from tracker IDs.

29. Target Identity Mapping

The mapping is:

```

Application Target ID
      |
      v
Current Tracker ID

```

Example:

```

Target 1 -> Tracker 7
Target 2 -> Tracker 11
Target 3 -> Tracker 15

```

After a tracker ID change:

```

Target 1 -> Tracker 21

```

The application identity remains:

```

Target 1

```

This is the fundamental distinction between tracking implementation state and application identity.

30. Target Lifecycle

A target may transition as follows:

```

CANDIDATE
  |
  | confirmed
  v
ACTIVE

```

```

      |
      | temporarily missing
      v
TEMPORARILY_LOST
      |
      +----- same track -----> ACTIVE
      |
      +----- successful re-ID -> REACQUIRED
      |
      +----- identity unrecoverable
      v
PERMANENTLY_LOST

```

PERMANENTLY_LOST shall not release the target slot for another dog.

For example:

```

Target 1 = Dog A
Target 2 = Dog B
Target 3 = Dog C

```

If Dog B disappears permanently:

```

Target 2 = PERMANENTLY_LOST

```

A later Dog D does not become Target 2.

This preserves the first-N requirement.

31. Target Re-Association

Re-association is required when:

```

old tracker ID disappears
      +
new tracker ID appears

```

The target manager should evaluate whether the new observation could correspond to a temporarily lost target.

Potential signals include:

- bounding-box proximity;
- motion prediction;
- tracker state;
- appearance/ReID information;
- temporal distance;
- association score.

The exact implementation should depend on the selected tracker.

The manager shall never perform arbitrary reassignment solely because a new dog appeared.

If confidence is insufficient:

```
old target -> PERMANENTLY_LOST  
new tracker -> potential new candidate
```

If all target slots have already been consumed, the new dog remains unselected.

32. ByteTrack Implementation Option

ByteTrack may be implemented behind the same tracker interface.

Its primary role in V1 is a lower-complexity alternative.

It is particularly suitable for:

```
static camera  
limited number of dogs  
short occlusions  
CPU-oriented processing
```

The application must not contain ByteTrack-specific decisions outside the tracker adapter.

33. BoT-SORT Implementation

BoT-SORT shall be the initial primary tracker implementation.

Configuration shall determine whether ReID is enabled.

Conceptually:

```
if config.TRACKER_TYPE == "botsort":  
    tracker = UltralyticsBoTSORTTracker(...)  
elif config.TRACKER_TYPE == "bytetrack":  
    tracker = UltralyticsByteTrackTracker(...)  
else:  
    raise ConfigurationError(...)
```

This conditional belongs in dependency construction, not throughout the processing pipeline.

34. ReID Configuration

ReID shall be optional.

Initial configuration:

```
REID_ENABLED = False
```

A later benchmark may establish that ReID is required for difficult scenes.

Enabling ReID should not alter the application-level target API.

The rest of the application should continue to see:

```
Tracker
TrackObservation
TargetObject
```

35. Main Processing Pipeline

src/processing/pipeline.py

The processing pipeline shall be the central frame-level algorithm.

Conceptual interface:

```
class ProcessingPipeline:
    def process(self) -> ProcessingResult:
        ...
```

The pipeline receives dependencies through its constructor.

Example:

```
ProcessingPipeline(
    reader=reader,
    detector=detector,
    tracker=tracker,
    selector=selector,
    target_manager=target_manager,
    renderer=renderer,
    writer=writer,
    performance_monitor=performance_monitor,
)
```

36. Processing Algorithm

The main algorithm is:

```
initialize
  |
  v
open reader
  |
  v
open writer
  |
  v
initialize detector
  |
  v
initialize tracker
  |
  v
loop
  |
  +--> read frame
```

```

|
+--> validate
|
+--> detect
|
+--> filter dogs
|
+--> track
|
+--> update target manager
|
+--> select new targets
|
+--> render
|
+--> write
|
+--> update metrics
|
+--> report progress
|
v
EOF
|
v
finalize writer

```

37. Detailed Frame Algorithm

Conceptually:

```
frame_index = 0
```

```
while not cancelled:
```

```
    success, frame = reader.read()
```

```
    if not success:
        handle_read_result()
        continue_or_finish()
```

```
    if not is_valid_frame(frame):
        handle_invalid_frame()
        continue
```

```
    frame_index += 1
```

```
    detections = detector.detect(frame)
```

```
    dog_detections = filter_dog_detections(detections)
```

```
    observations = tracker.update(
        frame,
        dog_detections,
    )
```

```
    target_manager.update_existing_targets(
        observations,
        frame_index,
    )
```

```

confirmed_candidates = selector.evaluate(
    observations,
    frame_index,
    target_manager,
)

for candidate in confirmed_candidates:
    if target_manager.target_count() >= config.MAX_TARGETS:
        break

    target_manager.create_target(candidate)

target_manager.finalize_visibility(
    observations,
    frame_index,
)

output_frame = renderer.render(
    frame,
    target_manager.targets(),
)

writer.write(output_frame)

performance_monitor.record_frame(...)

report_progress(...)

```

The exact ordering between target update and selection shall be implemented carefully so that a tracker observation already mapped to an existing target cannot create another target.

38. Detection and Tracking Ordering

The tracker should receive the current frame and current dog detections.

The selector should operate on tracker observations rather than raw detections.

Therefore:

```

raw YOLO detection
    |
    v
dog detection
    |
    v
tracker
    |
    v
TrackObservation
    |
    +--> existing target update
    |
    +--> candidate selection

```

This ensures that target selection is based on temporal object evidence rather than isolated detector outputs.

39. Rendering Design

src/rendering/renderer.py

Conceptual interface:

```
class Renderer(Protocol):
    def render(
        self,
        frame: np.ndarray,
        targets: list[TargetObject],
    ) -> np.ndarray:
        ...
```

The renderer shall:

1. Make a writable output frame.
2. Iterate over currently visible selected targets.
3. Draw bounding boxes.
4. Draw class/confidence labels.
5. Return the annotated frame.

It shall not invoke the detector or tracker.

40. Label Construction

Normal V1 output:

dog 0.87

Conceptually:

```
label = f"{target.class_name} {confidence:.2f}"
```

If the target currently has no detector observation, no fabricated confidence should be displayed.

Debug mode may optionally use:

Target 2 | dog 0.87

but normal output should remain:

dog 0.87

41. Temporarily Lost Targets

A temporarily lost target should not receive a fabricated current bounding box.

Therefore:

ACTIVE

-> visible box

TEMPORARILY_LOST

-> no current box

Optional debug visualization may show the last-known location, but this should be explicitly marked as stale if implemented.

42. Video Writer

src/video/writer.py

Conceptual interface:

```
class VideoWriter(Protocol):
    def open(
        self,
        path: Path,
        metadata: VideoMetadata,
    ) -> None:
        ...

    def write(self, frame: np.ndarray) -> None:
        ...

    def close(self) -> None:
        ...
```

The implementation shall wrap:

cv2.VideoWriter

Writer initialization must be checked.

A writer that failed to initialize shall not silently accept frames.

43. Output Codec

The writer configuration shall define the selected output format.

The implementation should test the requested codec before processing a long video.

A writer failure shall terminate processing because an incomplete or invalid video is not a valid V1 result.

44. Temporary Output Manager

src/output/temporary_output.py

The manager shall create a session-specific temporary directory.

Conceptually:

```
temporary/  
  session-<unique-id>/  
    processed.mp4
```

Python's `tempfile` facilities should be used rather than manually constructing predictable temporary filenames.

The manager shall expose:

```
create()  
path()  
cleanup()
```

Cleanup shall be idempotent.

45. Processing Result

The processing pipeline should return an application-level result.

Example:

```
@dataclass(frozen=True)  
class ProcessingResult:  
    success: bool  
    output_path: Path | None  
    frames_processed: int  
    processing_time: float  
    average_processing_fps: float | None  
    targets_selected: int
```

An error result may additionally contain an application error code.

This prevents the GUI from having to inspect internal exceptions to determine the result.

46. Playback Design

`src/gui/playback.py`

The playback controller shall consume the completed temporary output.

It should not require access to:

```
Detection  
TrackObservation  
TargetObject  
YOLO  
Tracker
```

Its responsibility is simply:

```
temporary video
  |
  v
playback
  |
  v
GUI
```

Playback failure should be treated independently from processing failure.

47. Save Manager

src/output/save_manager.py

Conceptual interface:

```
class SaveManager:
    def save(
        self,
        temporary_path: Path,
        destination_path: Path,
    ) -> None:
        ...
```

The Save Manager shall:

1. Verify the temporary output exists.
 2. Validate the destination.
 3. Apply overwrite policy.
 4. Check available disk space where practical.
 5. Copy the completed result.
 6. Verify the destination exists and has a valid size.
 7. Report success or failure.
-

48. Save Operation

The GUI should initiate saving.

Sequence:

```
User presses Save
  |
  v
Open destination dialog
  |
  v
User selects destination
  |
  v
```

```
Validate destination
  |
  v
Copy temporary output
  |
  v
Verify saved file
  |
  v
Report success
```

The temporary file remains the playback source.

49. Save Failure

If saving fails:

temporary result remains intact

The GUI enters:

SAVE_FAILED

The user may retry without rerunning detection.

This is important because saving is separate from processing.

50. Explicit Permanent Storage Rule

The application shall never automatically convert:

temporary output

into:

permanent user output

Processing completion alone does not constitute permission to permanently save the result.

The only permanent-save transition is:

User presses Save

followed by a successful save operation.

51. Performance Monitor

src/monitoring/performance.py

The performance monitor shall maintain aggregate measurements.

Conceptual API:

```
class PerformanceMonitor:
    def start(self) -> None:
        ...

    def record_frame(
        self,
        inference_time: float,
    ) -> None:
        ...

    def snapshot(self) -> PerformanceSnapshot:
        ...

    def finish(self) -> PerformanceSnapshot:
        ...
```

52. Performance Metrics

The implementation shall distinguish:

- Source FPS
- Processing FPS
- Inference latency
- Total processing time
- CPU usage
- Memory usage
- GPU usage

Processing FPS:

$\text{processed_frames} / \text{processing_elapsed_time}$

Inference latency:

$\text{inference_end} - \text{inference_start}$

The two measurements must not be conflated.

53. Inference Timing

The detector call shall be timed at the detector boundary.

Conceptually:

```
start = time.perf_counter()
detections = detector.detect(frame)
elapsed = time.perf_counter() - start
```

This measurement should be passed to the performance monitor.

The detector itself does not need to perform logging.

54. Performance Aggregation

The application shall not log every frame.

Instead:

5-second aggregation interval

may be used as the initial configuration.

Example:

```
Performance:
frames=152
processing_fps=30.4
avg_inference_ms=25.8
cpu=72%
memory=1.4 GB
gpu=64%
```

At completion, a final aggregate shall be logged.

55. Warm-Up Handling

The implementation may mark the first configured number of inference calls as warm-up.

For example:

```
WARMUP_FRAMES = 5
```

Warm-up measurements can either be:

excluded from steady-state statistics

or:

reported separately

The chosen policy shall remain consistent across benchmarks.

56. CPU and Memory Monitoring

`psutil` may be sampled at the performance interval rather than every frame.

For example:

```
every 5 seconds:
  CPU
  RSS memory
```

This minimizes monitoring overhead.

CPU and memory metrics are diagnostic measurements and should not cause processing to fail if unavailable.

57. GPU Monitoring

GPU utilization shall be optional.

If reliable GPU metrics cannot be obtained:

GPU = unavailable

The processing pipeline shall continue.

GPU metric collection must never become a functional dependency of detection.

58. Logging Design

src/logging/logger.py

A centralized logger should be initialized at application startup.

Modules should obtain named loggers:

```
logger = logging.getLogger(__name__)
```

Recommended levels:

```
DEBUG
INFO
WARNING
ERROR
```

TRACE is not required in V1.

59. Logging Examples

Startup:

```
INFO Model loaded successfully
INFO Input video: example.mp4
INFO Source resolution: 1920x1080
INFO Source FPS: 30.0
INFO Tracker: botsort
INFO ReID: disabled
```

Target selection:

```
INFO Target 1 selected from tracker 7
```

INFO Target 2 selected from tracker 11

Temporary loss:

WARNING Target 2 temporarily lost

Reassociation:

INFO Target 2 reacquired with tracker 18

Identity ambiguity:

WARNING Unable to confidently re-associate lost target 2

60. Bounded Repeated Error Logging

Repeated errors shall not flood the log.

A reusable failure counter may track:

read failures
inference failures
tracker failures

The logging policy may be:

first failure -> log
subsequent failures -> aggregate
threshold reached -> log ERROR and terminate

Example:

WARNING Frame read failure
WARNING 5 consecutive frame-read failures
ERROR Maximum consecutive frame-read failures exceeded

The implementation shall not contain an infinite loop that only logs the same failure.

61. Error Classes

A small hierarchy may be used:

```
class ApplicationError(Exception):  
    pass  
  
class InputValidationError(ApplicationError):  
    pass  
  
class VideoReadError(ApplicationError):  
    pass
```

```
class VideoWriteError(ApplicationError):
    pass

class ModelLoadError(ApplicationError):
    pass

class InferenceError(ApplicationError):
    pass

class TrackerError(ApplicationError):
    pass

class ConfigurationError(ApplicationError):
    pass

class SaveError(ApplicationError):
    pass
```

The hierarchy should remain small.

There is no need for dozens of specialized exception types.

62. Error Propagation

Low-level components should raise meaningful application exceptions.

The controller catches these and translates them into:

```
technical log
+
user-readable GUI message
```

For example:

```
Internal:
ModelError: file does not exist
```

```
GUI:
Required YOLO model is unavailable.
```

The traceback belongs in the log, not the normal GUI.

63. Cancellation

Cancellation shall be cooperative.

A thread-safe event can be used:

```
threading.Event()
```

The worker checks:

```
if cancellation_event.is_set():
    stop_processing()
```

The check should occur at a safe frequency, normally once per frame.

The application shall not forcibly terminate the worker thread.

64. Cancellation Cleanup

Cancellation sequence:

```
cancel requested
  |
  v
worker observes cancellation
  |
  v
stop frame loop
  |
  v
close writer
  |
  v
close tracker
  |
  v
close detector
  |
  v
close reader
  |
  v
delete temporary output
  |
  v
GUI -> IDLE
```

Partial output shall be treated as invalid in V1.

65. Resource Management

Each resource-owning component should expose an explicit `close()`.

The controller should use a centralized cleanup path.

Conceptually:

```
def cleanup():
    safely_close(writer)
    safely_close(tracker)
    safely_close(detector)
    safely_close(reader)
    safely_cleanup_temp_output()
```

Each cleanup operation should be safe to call more than once.

66. Cleanup Failure Policy

A cleanup failure shall be logged but should not normally replace the original processing error.

For example:

Original:
VideoWriter failed

Cleanup:
temporary file deletion failed

The user should primarily be told:

Video processing failed.

while both details are retained in the log.

67. Processing Thread Ownership

The processing worker owns:

VideoReader
Detector
Tracker
TargetSelector
TargetManager
Renderer
VideoWriter
PerformanceMonitor

The GUI owns:

widgets
playback UI
dialogs
GUI state

The GUI should not manipulate processing objects directly while processing is active.

68. Dependency Injection

V1 does not require a dependency-injection framework.

Simple constructor injection is sufficient.

Example:

```
pipeline = ProcessingPipeline(
```

```
    reader=reader,  
    detector=detector,  
    tracker=tracker,  
    selector=selector,  
    target_manager=target_manager,  
    renderer=renderer,  
    writer=writer,  
)
```

This provides enough isolation for testing without introducing an enterprise DI container.

69. Preprocessing

V1 preprocessing should be minimal.

The detector should receive the frame format expected by the selected Ultralytics API.

Potential operations include:

```
resize  
color conversion  
normalization
```

Only operations that provide measurable value should be added.

Unnecessary conversions such as:

```
BGR -> RGB -> BGR
```

shall be avoided.

70. Frame Resolution

The implementation may expose a configurable detector input size if supported by the selected YOLO API.

The original video dimensions should remain the output dimensions unless explicitly designed otherwise.

Therefore:

```
Input frame: 1920x1080  
    |  
    +--> detector may internally resize  
    |  
    v  
Renderer/output: 1920x1080
```

This preserves output quality while allowing inference-size optimization.

71. Progress Calculation

If:

```
frame_count_known == True
```

and the frame count is reliable:

```
progress = frame_index / frame_count * 100
```

Otherwise the GUI should display:

```
Processing...
```

or an indeterminate progress indicator.

The implementation shall not fabricate progress based on unreliable metadata.

72. Processing Completion

Successful completion requires:

```
EOF reached  
AND  
all required frames written  
AND  
writer successfully finalized
```

Then:

```
temporary output exists
```

The GUI transitions to:

```
COMPLETED
```

and enables playback and Save.

73. Zero-Target Scenario

It is possible that the video contains:

```
zero qualifying dogs
```

This is not necessarily an application failure.

The result should still be a valid processed video if:

```
video decoded successfully  
+  
processing completed
```

The GUI may report:

Processing complete. No qualifying dogs were detected.

The output remains available for playback and explicit saving.

74. Fewer Than N Dogs

If:

MAX_TARGETS = 3

but only two distinct dogs appear:

Target 1

Target 2

The application completes normally.

It does not need to manufacture a third target.

75. More Than N Dogs

If:

MAX_TARGETS = 3

and five distinct dogs appear:

Dog A -> Target 1

Dog B -> Target 2

Dog C -> Target 3

Dog D -> ignored

Dog E -> ignored

Dog D and Dog E must not replace the existing targets.

76. Target Selection Example

Assume:

MAX_TARGETS = 2

TARGET_CONFIRMATION_FRAMES = 3

Video sequence:

Frame 1:

Dog A

Frame 2:

Dog A

Frame 3:
Dog A + Dog B

Frame 4:
Dog A + Dog B

Frame 5:
Dog A + Dog B

Frame 6:
Dog C

Possible result:

Dog A -> Target 1
Dog B -> Target 2
Dog C -> ignored

Even if Dog A later disappears, Dog C must not become Target 1.

77. Simultaneous Candidates Example

Suppose two dogs become confirmed in the same frame:

Dog A center_x = 300
Dog B center_x = 900

The selector assigns:

Dog A -> Target 1
Dog B -> Target 2

because the defined deterministic ordering is left-to-right.

If their positions are equal within the chosen representation:

tracker ID

acts as the tie-breaker.

78. Identity-Switch Protection

The target manager must not perform:

```
target.tracker_id = new_tracker_id
```

merely because a different tracker observation appears near the target.

Reassignment requires sufficient evidence.

Otherwise:

Dog A disappears
Dog B appears in same location

could incorrectly produce:

Target 1 = Dog B

The correct behavior is to preserve the identity reservation until re-association is sufficiently supported or mark the target permanently lost.

79. Tracker Failure Policy

If the tracker fails repeatedly, the application must not silently revert to detector-only processing.

Tracking is a functional V1 requirement.

Therefore:

```
tracker unavailable
  |
  v
processing failure
```

rather than:

```
tracker unavailable
  |
  v
continue without identity
```

This prevents the generated video from falsely appearing to satisfy persistent-tracking requirements.

80. Video Writer Failure Policy

A writer failure invalidates the processing result.

The pipeline shall stop.

It shall not continue processing hundreds or thousands of frames if the output cannot be written.

Sequence:

```
writer failure
  |
  v
stop pipeline
  |
  v
cleanup
  |
  v
delete invalid temporary output
  |
  v
GUI -> FAILED
```

81. Model Loading Failure

Model loading shall happen before frame processing.

If:

`MODEL_PATH` does not exist

the application shall fail during initialization.

It should not repeatedly attempt to load the model for every frame.

82. Device Selection

Configuration:

`DEVICE = auto`

may select an available supported accelerator.

Explicit:

`DEVICE = cuda`

requires CUDA availability.

If unavailable:

`ModelLoadError`

The application shall not silently change explicit CUDA configuration to CPU.

83. Dependency Version Handling

The implementation shall pin tested package versions.

The application should log relevant runtime versions at startup:

```
Python version
OpenCV version
Ultralytics version
PyTorch version
PySide6 version
```

This makes performance and model behavior reproducible.

84. Testing Architecture

The testing structure shall correspond to component boundaries.

```
test/
├── unit/
│   ├── test_detection.py
│   ├── test_selection.py
│   ├── test_target_manager.py
│   ├── test_pipeline.py
│   ├── test_video_validation.py
│   └── test_performance.py
├── integration/
│   ├── test_detector_adapter.py
│   ├── test_tracker_adapter.py
│   └── test_processing_pipeline.py
├── gui/
│   ├── test_main_window.py
│   └── test_gui_states.py
├── performance/
│   └── test_processing_performance.py
└── e2e/
    └── test_full_video_processing.py
```

85. Unit Test: Dog Filtering

Input:

```
person 0.91
dog 0.83
cat 0.88
dog 0.31
```

With:

```
TARGET_CLASS = dog
DETECTION_CONFIDENCE = 0.40
```

Expected:

```
dog 0.83
```

Only.

86. Unit Test: First-N Selection

Configuration:

```
MAX_TARGETS = 2
```

Input:

```
confirmed tracker 1
confirmed tracker 2
confirmed tracker 3
```

Expected:

Target 1 -> tracker 1
Target 2 -> tracker 2
tracker 3 -> unselected

87. Unit Test: Candidate Confirmation

Configuration:

TARGET_CONFIRMATION_FRAMES = 3

Input:

Frame 1 -> tracker 7
Frame 2 -> tracker 7

Expected:

no target

Then:

Frame 3 -> tracker 7

Expected:

target created

88. Unit Test: False-Positive Prevention

Input:

Frame 1 -> tracker 5
Frame 2 -> tracker 5 disappears

Expected:

target count = 0

A later real dog can still consume the first target slot.

89. Unit Test: Duplicate Prevention

Input:

Target 1 -> tracker 7

Later:

tracker 7 -> same dog

Expected:

target count remains 1

No second target shall be created.

90. Unit Test: Tracker ID Change

Initial:

Target 1 -> tracker 7

Later:

tracker 7 disappears
tracker 15 appears

If re-association succeeds:

Target 1 -> tracker 15

The application target ID remains:

Target 1

91. Unit Test: Permanent Loss

If re-association fails:

Target 1 -> PERMANENTLY_LOST

and a new dog appears:

Dog D -> candidate

Expected:

Dog D does not replace Target 1

92. Unit Test: Simultaneous Selection

Two candidates become confirmed in one frame.

Expected ordering:

leftmost -> lower target ID
rightmost -> next target ID

Tie-breaking shall be deterministic.

93. Integration Testing

Integration tests should verify real component interactions without necessarily requiring a real YOLO model.

Example:

```
FakeVideoReader
+
FakeDetector
+
FakeTracker
+
RealTargetSelector
+
RealTargetManager
+
FakeRenderer
+
FakeWriter
```

This allows the actual target-selection and identity logic to be tested deterministically.

94. Fake Detector

Example behavior:

```
Frame 1 -> Dog A
Frame 2 -> Dog A
Frame 3 -> Dog A + Dog B
Frame 4 -> Dog A + Dog B
Frame 5 -> Dog A + Dog B
```

The fake detector should return real `Detection` objects.

No Ultralytics dependency is needed.

95. Fake Tracker

The fake tracker should return predetermined observations.

Example:

```
Frame 1 -> tracker 7
Frame 2 -> tracker 7
Frame 3 -> tracker 7 + tracker 11
Frame 4 -> tracker 7 + tracker 11
Frame 5 -> tracker 7 + tracker 11
```

This makes identity tests deterministic.

96. GUI Testing

GUI tests should verify:

IDLE
FILE_SELECTED
VALIDATING
PROCESSING
COMPLETED
FAILED
SAVING
SAVE_FAILED

and button states.

For example:

IDLE:
Process disabled

FILE_SELECTED:
Process enabled

PROCESSING:
Save disabled

COMPLETED:
Save enabled
Playback enabled

97. End-to-End Test

A small deterministic video shall be used.

The end-to-end test should verify:

input opens
model initializes
processing completes
output exists
output is readable
dogs are annotated
target count is correct

The test should not depend on arbitrary behavior from a large uncontrolled real-world video.

98. Error Injection Testing

Tests shall deliberately inject:

- missing video
- unsupported extension
- unopenable video
- zero-frame video
- invalid frame
- reader failure
- model load failure
- inference exception
- tracker exception
- writer failure
- save failure
- cancellation
- unexpected exception

Each test shall verify both:

- correct application state
- +
- correct resource cleanup

99. Performance Test Design

Representative videos should include:

- short video
- long video
- single dog
- multiple dogs
- similar dogs
- occlusion
- high resolution

At minimum, benchmark:

- CPU
- GPU where available

Metrics:

- processing FPS
- inference latency
- total processing time
- CPU
- memory
- GPU where available

Performance targets should be established from actual benchmark results rather than embedded assumptions.

100. Long-Running Stability Test

A long video should be processed repeatedly to detect:

- memory growth

resource leaks
tracker degradation
performance degradation
temporary-file problems
log flooding

Memory should be sampled throughout processing.

The application should not retain every processed frame or every historical detection.

101. Memory Management

The pipeline should process one frame at a time.

Conceptually:

```
read frame
  |
  v
process frame
  |
  v
write frame
  |
  v
release/reuse frame
  |
  v
next frame
```

The application shall not accumulate the complete input video in memory.

Likewise, it should not maintain an unbounded detection history.

102. Detection History

V1 only needs current and target-related state.

It does not require:

all detections from all frames

The target manager should retain only the state necessary for identity and tracking.

This keeps memory usage bounded.

103. Configuration Validation

Configuration should be validated before processing.

Examples:

```
if config.MAX_TARGETS < 1:
    raise ConfigurationError(...)

if not 0.0 <= config.DETECTION_CONFIDENCE <= 1.0:
    raise ConfigurationError(...)

if config.TARGET_CONFIRMATION_FRAMES < 1:
    raise ConfigurationError(...)
```

Tracker-specific configuration should be validated by the tracker adapter.

104. Output Validation

Before playback or Save is enabled, the application should verify:

```
temporary file exists
file size > 0
writer finalized successfully
output can be reopened
```

If the output cannot be reopened, processing should be considered failed or the result marked invalid.

105. Application Startup Sequence

Startup:

```
main()
|
v
initialize logging
|
v
validate configuration
|
v
create Qt application
|
v
create main window
|
v
show GUI
```

The YOLO model does not necessarily need to be loaded at application startup.

It may be loaded when processing begins so that opening the GUI remains lightweight.

106. Processing Initialization Sequence

When the user selects Process:

```
validate input
  |
  v
create temporary output
  |
  v
open reader
  |
  v
read/validate initial frame
  |
  v
load detector
  |
  v
initialize tracker
  |
  v
initialize writer
  |
  v
start performance monitor
  |
  v
start processing loop
```

If any initialization stage fails, already-created resources must be cleaned up.

107. Processing Shutdown Sequence

Normal completion:

```
EOF
  |
  v
final frame confirmed written
  |
  v
close writer
  |
  v
close tracker
  |
  v
close detector
  |
  v
close reader
  |
  v
finalize metrics
  |
  v
validate temporary output
  |
  v
notify GUI
```

108. Failure Shutdown Sequence

Failure:

```
exception
|
v
stop processing
|
v
log exception
|
v
close writer
|
v
close tracker
|
v
close detector
|
v
close reader
|
v
delete invalid temporary output
|
v
notify GUI
```

The cleanup path shall be shared as much as practical with normal completion.

109. Save Lifecycle

```
COMPLETED
|
| Save
v
SAVING
|
+----- success ---> COMPLETED
|
+----- failure ---> SAVE_FAILED
```

A save failure must not invalidate the temporary processing result.

110. Logging Lifecycle

Logging starts before significant application initialization.

It should record:

```
startup
configuration
```

input
model
tracker
processing
target events
performance
errors
shutdown

Logging should be flushed/closed during application shutdown.

111. Thread Safety

The processing worker owns mutable tracking state.

The GUI should receive immutable snapshots/events rather than directly accessing mutable tracker state.

For example:

```
Worker:
TargetObject state
  |
  v
TargetSnapshot
  |
  v
Qt signal
  |
  v
GUI
```

The GUI therefore does not need locks around target state.

112. Avoiding Excessive Concurrency

V1 should use one processing worker.

The following are not required:

multiple inference workers
frame queues
distributed workers
producer/consumer pipelines
GPU worker pools
message brokers

Sequential frame processing is simpler and preserves tracker continuity.

113. Thread Cancellation Safety

Cancellation shall occur between frame-processing operations.

The worker should not abandon the pipeline in the middle of:

```
writer.write()
```

or another resource-sensitive operation.

The preferred boundary is:

```
finish current frame
  |
  v
check cancellation
  |
  v
exit loop
```

114. Testing Resource Cleanup

Mock resource classes should record whether:

```
close()
```

was called.

Tests shall verify cleanup for:

```
success
failure
cancellation
unexpected exception
```

For example:

```
reader.closed == True
writer.closed == True
tracker.closed == True
detector.closed == True
```

115. Temporary File Cleanup Tests

Tests should verify:

```
successful session -> temporary file exists during playback
application shutdown -> temporary file removed
failed processing -> temporary file removed
cancelled processing -> temporary file removed
save failure -> temporary file retained
```

This directly tests the temporary-first output requirement.

116. Security-Oriented Implementation Rules

The application is offline, so there is no network attack surface in V1.

Nevertheless:

- user-selected paths shall be validated;
 - temporary files shall use secure temporary-file mechanisms;
 - logs shall not contain frame data;
 - model paths shall be validated;
 - arbitrary network URLs shall be rejected;
 - output paths shall be handled through standard filesystem APIs.
-

117. File Extension Handling

The application should normalize suffix comparisons:

```
suffix = path.suffix.lower()
```

Then compare against:

```
config.SUPPORTED_EXTENSIONS
```

This prevents:

```
VIDEO.MP4
```

from being incorrectly rejected.

Extension validation remains only a preliminary check. Actual decodability must still be tested.

118. Model Path Handling

The model path should be resolved consistently.

Example:

```
MODEL_PATH = Path(...)
```

The implementation should avoid depending on the current working directory when practical.

A project-root-aware configuration is preferable.

119. Logging of User Paths

The full input path may be logged for diagnostics, but unnecessary video metadata or user content should not be copied into logs.

The application shall never log frame pixel arrays or complete detection histories.

120. Documentation Requirements

The implementation repository should document:

- installation
- dependency versions
- model provisioning
- configuration
- supported video formats
- tracker configuration
- running the application
- running tests
- performance benchmarking
- known limitations

The README should explicitly state that:

persistent tracking does not constitute guaranteed biometric/physical identity.

The tracker can lose identity under sufficiently difficult conditions.

121. Known Functional Limitation

The application cannot guarantee perfect physical identity through arbitrary occlusion or disappearance.

For example:

- Dog A disappears for a long period.
- Dog B appears.
- Dog A later returns.

Without sufficient appearance or temporal information, the system may not be able to prove that the returning dog is Dog A.

The implementation must therefore prefer:

PERMANENTLY_LOST

over:

incorrect identity substitution

when confidence is insufficient.

122. Important Identity Invariant

The following invariant shall always hold:

A selected application target must never silently change from one physical dog to another dog.

This is more important than maximizing the number of continuously visible target boxes.

If identity becomes uncertain, the target remains reserved and may be marked lost.

123. Important First-N Invariant

The following invariant shall also hold:

Once MAX_TARGETS target slots have been assigned, no later dog can replace an existing target.

The only exception would be an explicit future product requirement changing the first-N semantics. Such behavior is not permitted in V1.

124. Important Confidence Invariant

The implementation must preserve the distinction:

Detection confidence
!=
Association confidence
!=
Target identity confidence

A high detector confidence does not prove that a detection is the same dog as a previous target.

Likewise, a strong tracker association does not necessarily imply a high detector confidence.

125. Important State Invariant

A target's application ID is immutable after creation.

For example:

Target 2

must remain:

Target 2

until the processing session ends.

Its tracker ID may change.

Its visibility may change.

Its lifecycle state may change.

Its application identity must not.

126. Component Dependency Direction

Dependencies should generally flow toward abstractions.

```
GUI
|
v
Application
|
v
Processing Pipeline
|
+--> Detector interface
+--> Tracker interface
+--> Video interfaces
+--> Renderer interface
```

Concrete implementations sit behind those interfaces.

For example:

```
UltralyticsDetector
      implements
Detector
```

and:

```
OpenCVVideoReader
      implements
VideoReader
```

127. What Should Not Be Abstracted

The implementation should not create abstractions for every small operation.

For example, these do not require independent interfaces:

```
DogFilterFactory
FrameValidationStrategyFactory
ConfigurationProviderFactory
TargetLabelBuilderFactory
```

unless a real future requirement justifies them.

The useful abstraction boundaries are:

Detector

Tracker
VideoReader
VideoWriter
Renderer
SaveManager

plus application state/data models.

128. Recommended Class Responsibility Summary

Application
Owns lifecycle and dependency coordination.

ProcessingPipeline
Owns frame-by-frame processing sequence.

VideoReader
Owns video input.

VideoWriter
Owns output encoding.

Detector
Owns model inference.

UltralyticsDetector
Owns Ultralytics-specific implementation.

Tracker
Owns temporal association.

UltralyticsTracker
Owns Ultralytics tracker implementation.

TargetSelector
Owns first-N selection policy.

TargetManager
Owns application target identity/state.

Renderer
Owns visual annotation.

PerformanceMonitor
Owns performance measurement.

SaveManager
Owns explicit permanent saving.

MainWindow
Owns GUI presentation and interaction.

PlaybackController
Owns playback behavior.

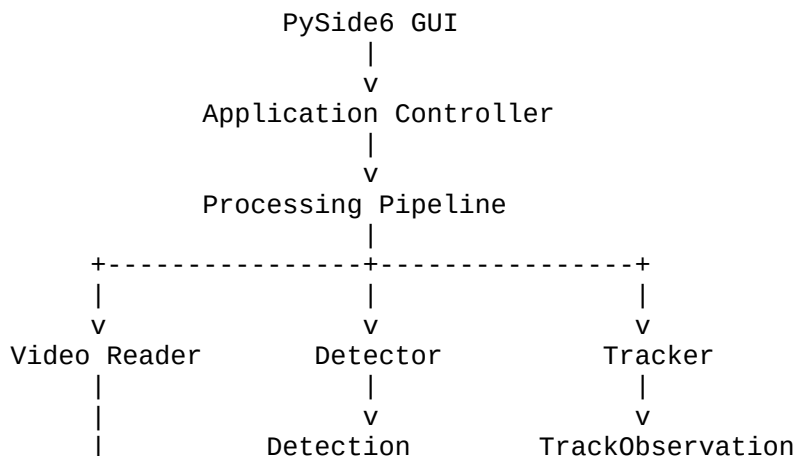
129. V1 Processing Sequence Summary

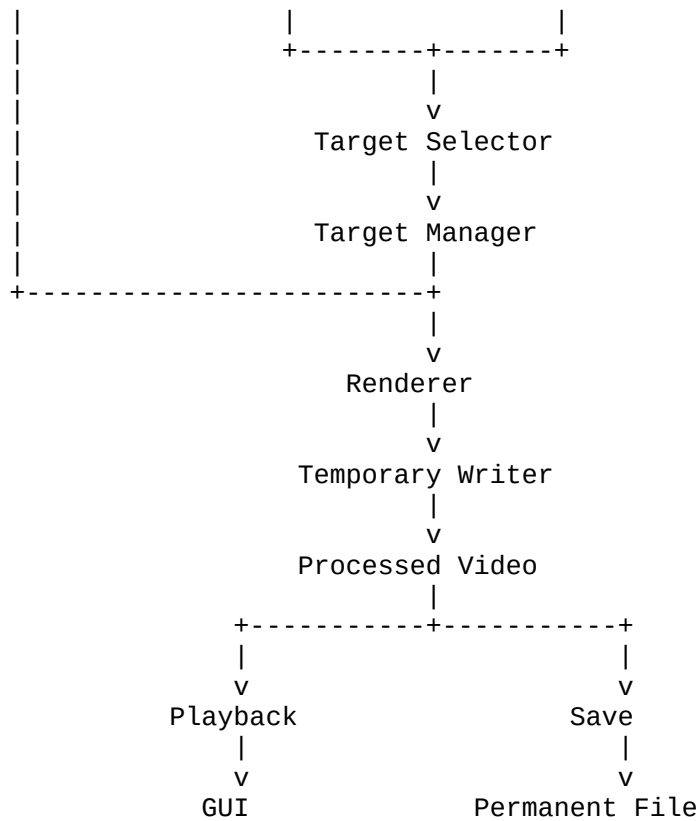
The complete implementation sequence is:

1. User selects video
 2. GUI validates selection
 3. Controller validates file
 4. Reader opens video
 5. Initial frame is validated
 6. Metadata is collected
 7. Temporary output is created
 8. YOLO model is loaded
 9. Tracker is initialized
 10. Writer is initialized
 11. Processing begins
 12. Frame is read
 13. Frame is validated
 14. YOLO inference occurs
 15. Results become Detection objects
 16. Non-dog detections are removed
 17. Low-confidence detections are removed
 18. Tracker updates
 19. TrackObservations are generated
 20. Existing targets are updated
 21. Candidates are evaluated
 22. Confirmed candidates consume remaining target slots
 23. Target state is updated
 24. Selected targets are rendered
 25. Frame is written
 26. Performance metrics are updated
 27. GUI progress is updated
 28. Next frame is processed
 29. EOF is reached
 30. Writer is finalized
 31. Resources are closed
 32. Output is validated
 33. GUI enables playback
 34. User optionally presses Save
 35. Temporary output remains until session cleanup
-

130. Final Implementation Architecture

The resulting V1 software structure is:





The core implementation principle is that each stage has one clear responsibility.

The detector determines what is visible. The tracker determines temporal correspondence. The target selector determines which dogs become the first N selected objects. The target manager owns persistent application identity. The renderer visualizes that state. The video components handle media, while the GUI handles presentation and user interaction.

This structure directly implements the SAD while keeping V1 constrained. The most important future extension points are the detector, tracker, target-selection policy, renderer, and video I/O interfaces. No database, network service, distributed architecture, or other infrastructure is necessary for the V1 implementation.